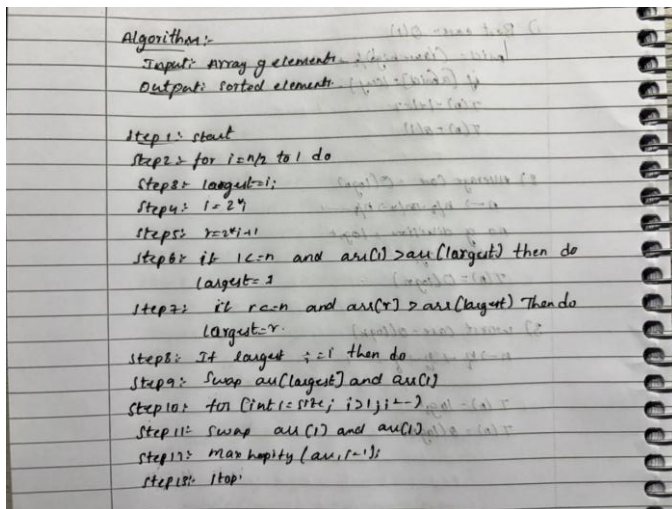


PRACTICAL-03

AIM: Implementation of max-heap sort algorithm.

1.) MAX HEAP

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void maxheapify(int arr[], int size, int i)
{
    int largest = i;
    int l = 2 * i;
    int r = 2 * i + 1;

    if (l <= size && arr[l] > arr[largest])
    {
        largest = l;
    }
}
```

```

    if (r <= size && arr[r] > arr[largest])
    {
        largest = r;
    }

    if (largest != i)
    {
        swap(arr[largest], arr[i]);
        maxheapify(arr, size, largest);
    }
}

void heapsort(int arr[], int size)
{
    for (int i = size / 2; i >= 1; i--)
    {
        maxheapify(arr, size, i);
    }
    for (int i = size; i > 1; i--)
    {
        swap(arr[1], arr[i]);
        maxheapify(arr, i - 1, 1);
    }
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n + 1];
    cout << "Enter elements: ";
    for (int i = 1; i <= n; i++)
    {

```

```

    cin >> arr[i];

}

heapsort(arr, n);

cout << "Sorted array: ";

for (int i = 1; i <= n; i++)

{

    cout << arr[i] << " ";

}

return 0;

}

```

OUTPUT:

```

PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA> cd "c:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA\" ; if ($?) { g++ maxheap.cpp -o maxheap } ; if ($?) { .\maxhe
ap }
Enter number of elements: 5
Enter elements: 23
4
21
43
17
Sorted array: 4 17 21 23 43
PS C:\Users\MANCHEM HIMESH\OneDrive\Desktop\DAA>

```

TIME COMPLEXITY:

Time complexity:

for ($i=n; i>1; i--$) $\rightarrow n$.

max heapify (arr, n, i) $\rightarrow \log n$.

for ($i=n; i>1; i--$) $\rightarrow n$

Swap ($arr[i], arr[n]$) $\rightarrow 1$

max heapify ($arr, i-1, 1$) $\rightarrow \log n$.

$T(n) = n \times \log n + n \times 1 \times \log n$

$= n \log n + n \log n = O(n \log n)$

Best case: $T(n) = O(n \log n)$

Average case: $T(n) = O(n \log n)$

Worst case: $T(n) = O(n \log n)$

Conclusion – Max Heap

A Max Heap is an efficient tree-based data structure in which the parent node is always greater than or equal to its children. It allows quick access to the maximum element, with $O(1)$ access and $O(n \log n)$ insertion and deletion. Max Heaps are widely used in priority queues, heap sort, and scheduling algorithms.